SQL Lab 2 – Table Joins

(requires access to the samples.high_peaks & samples.high_peaks_ascents tables)

In order to complete this lab, you will need to connect to the class database (INFO4555) via the SQL Developer application found on your AppStream. Details on how to connect to the class database may be found in document: *"Connecting to class database via SQL Developer",* in the project files folder of the class Canvas site.

If you completed SQL Lab 1, you were introduced to the "high_peaks" table, which is located in the "samples" schema of the INFO4555 database. As a review, the HIGH_PEAKS table contains a record for each of the 46 High Peaks of the Adirondack mountains of New York state.

HIGH_PEAKS		
Column Name	**Data Type**	**Description**
Peak_Rank	Number	Rank (1-46) of the peak by elevation
Peak_Name (PK)	Character	Name of Peak as designated by the 46ers club. This is also a unique identifier for this table.
Elevation	Number	Elevation of Peak measured in feet.
Ascent_of_climb	Number	The Total vertical ascent measured in feet.
Difficulty	Number	Difficulty rating (1-7) 1 being easiest, according to Adirondack.net
Round_Trip_mileage	Number	Round trip mileage of hike, according to Adirondack.net
Typical_Hike_Time_Hrs	Number	Typical hike time (round trip) in hours, according to Adirondack.net

For this lab we will be introducing a second table called "high_peaks_ascents", also located in the "samples" schema. The high_peaks_ascents table contains the names of hikers who have climbed all 46 of the Adirondack High Peaks. The Adirondack 46ers club was founded in 1918. To become a member of the 46ers club you must climb all 46 peaks. This high_peaks_ascents table contains all of the 46er members from 1918 – 2018, their names, along with a few other attributes described in the metadata below.

HIGH_PEAKS_ASCENTS		
Column Name	**Data Type**	**Description**
Member_id (PK)	character	Unique number assigned by the 46ers club. Indicates the order the member completed the 46 peaks. If a member number has a W suffix, the member has also climbed all 46 peaks during the winter months.
Member_name	character	Name of climber / member used on membership application.
First_peak	character	Name of the first ADK High Peak climbed by member.
First_ascent_date	date	Date of the member's first high peak ascent.
Last_peak	character	Name of the last ADK High Peak climbed by member.

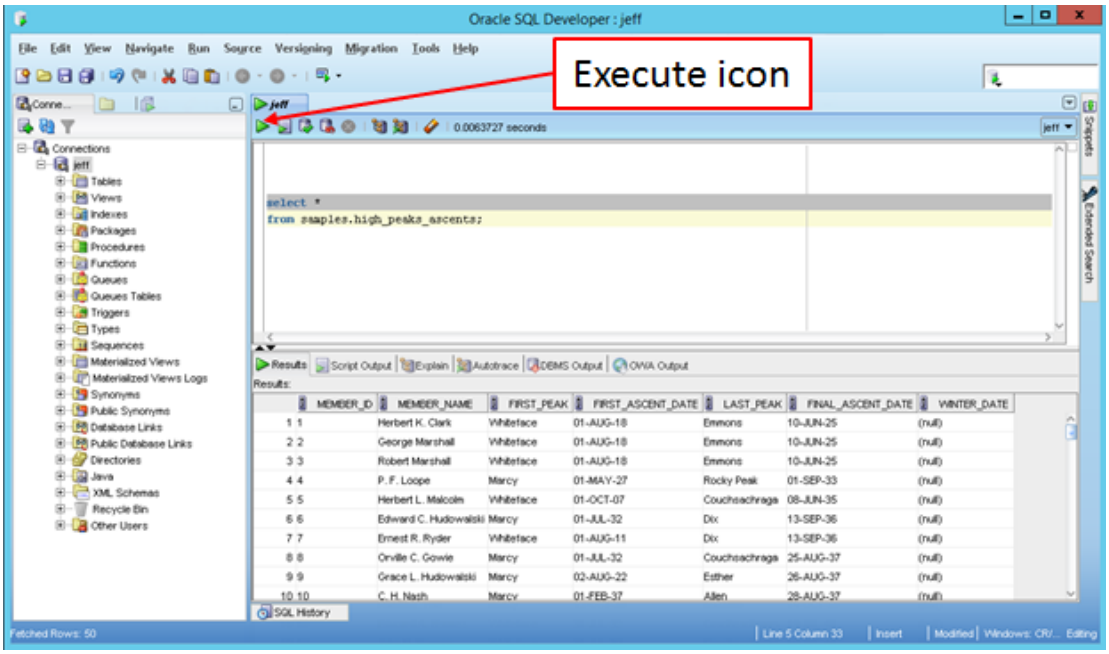
SQL Lab 2 – Table Joins

(requires access to the samples.high_peaks & samples.high_peaks_ascents tables)

Final_ascent_date	date	Date climber completed final 46th ascent.
Winter_date	date	If the member has also climbed all 46 peaks during the winter months, this attribute will contain the date the winter 46 were completed.

Let's start with a simple query of the high_peaks_ascents table.

```
SELECT *
FROM samples.high_peaks_ascents;
```



This will give you a feel for the data included in this table. Now let's do a little more analysis of the high_peaks_ascents table.

Let's see how many records the table contains.

```
SELECT count(*)
FROM samples.high_peaks_ascents;
```

Q1) How many records are in the high_peaks_ascents table?

Now let's see how many records have a null value for first_ascent_date.

```
SELECT *
FROM samples.high_peaks_ascents
WHERE first_ascent_date is null;
```

SQL Lab 2 – Table Joins

(requires access to the samples.high_peaks & samples.high_peaks_ascents tables)

This query will list the full records where the first_ascent_date is null. What if you want to count the number of records that have a null value for first_ascent_date? Try the following:

```
SELECT count(*)
FROM samples.high_peaks_ascents
WHERE first_ascent_date is null;
```

Q2) How many records contain a null value for first_ascent_date?

Note you can also test for records not containing a null value with “is not null”.

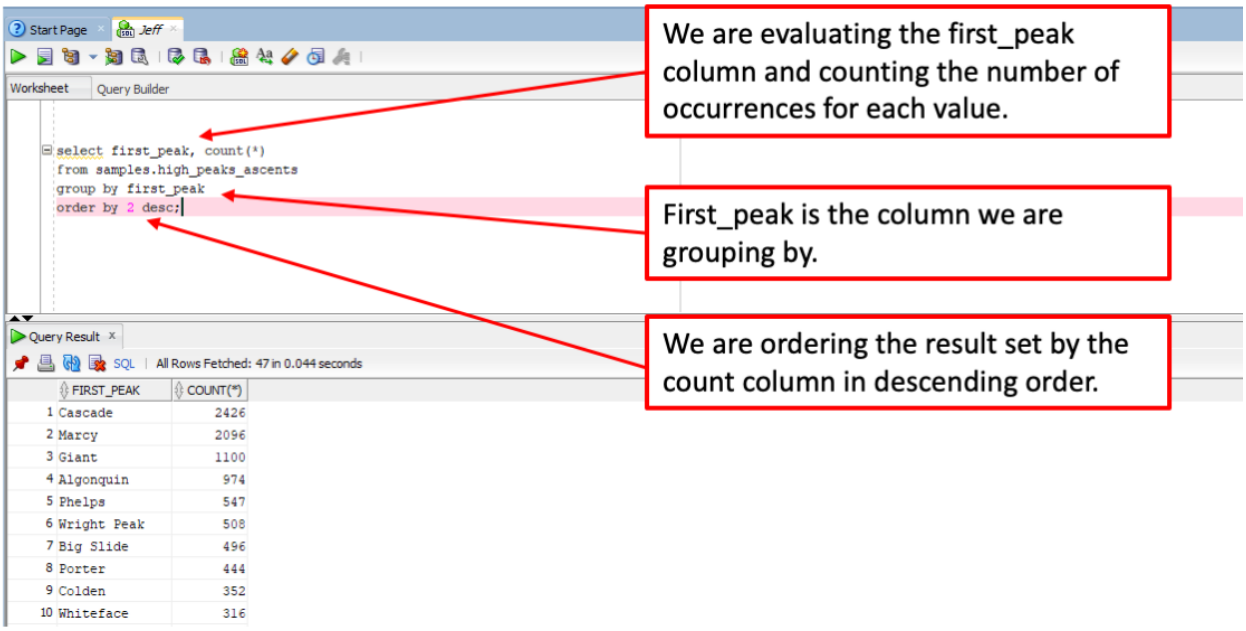
According to the data dictionary for the high_peaks_ascents table the winter_date column contains a date for any member who climbed all 46 peaks during the winter months, otherwise the column is null. Write and execute a query to determine how many 46er members completed the 46 during the winter.

Q3) How many members climbed the 46 peaks in the winter?

S3) paste your SQL into the Worksheet.

Let’s look at the distribution of values for the first_peak attribute.

```
select first_peak, count(*)
from samples.high_peaks_ascents
group by first_peak
order by 2 desc;
```



SQL Lab 2 – Table Joins

(requires access to the samples.high_peaks & samples.high_peaks_ascents tables)

Q4) What is the least common first_peak?

Q5) How many members climbed this peak as their first peak?

Now let's try joining the two high peaks tables. Let's join the high_peaks_ascents.first_peak column to the high_peaks.peak_name as a simple inner join.

```
select *
from samples.high_peaks_ascents a, samples.high_peaks hp
where a.first_peak = hp.peak_name;
```

Notice the use of table aliases: a and hp, this cuts out a lot of typing when specifying the joins.

Now count the number of records resulting from this inner join.

```
select count(*)
from samples.high_peaks_ascents a, samples.high_peaks hp
where a.first_peak = hp.peak_name;
```

Q6) What is the number of records this inner join returns?

Did you notice that there is a difference in the record count between the total records in the high_peaks_ascents table (Question 1) and the result set when we do an inner join between the high_peaks_ascents table and the high_peaks table. We lose 9 records in the result set, after the inner join. Apparently there are values in the high_peaks_ascents.first_peak column that don't exist in the high_peaks.peak_name column. To figure out what the value discrepancy is, we can query each table and do a minus between the two data sets as follows.

```
select first_peak
from samples.high_peaks_ascents
group by first_peak;
```

This query gives us a distinct list of values for the column first_peak. If you scroll down, you will notice there are 47 distinct values. Now try a similar query against the high_peaks table and peak_name column.

```
select peak_name
from samples.high_peaks
group by peak_name;
```

You will notice there are 46 distinct values for peak_name. This makes sense because there are only 46 peaks, however, we do have a mismatch. There is one more value in the high_peaks_ascents.first_peak

column, than in the high_peaks.peak_name column. To identify the additional value, we can use a minus operator between the two queries we just wrote.

```
select first_peak
from samples.high_peaks_ascents
group by first_peak
minus
select peak_name
from samples.high_peaks
group by peak_name;
```

The result of the above record is the value “Unknown”. Apparently, some of the members couldn’t remember which mountain they climbed first when they started their many year adventure. This may be okay for the ADK 46ers application process, but this “Unknown” value causes us to lose members when we perform our join.

When the driving table column, in the case the high_peaks_ascents.first_peak column in a join contains records that do not exist in the table and column being joined to, the result set will be restricted by the values found in the table being joined to. In some cases this may be the desired result. However, in this case, and in many cases you do not want to see all records from the driving table, whether there is a match in the join or not. For this scenario, you use an outer join.

```
select count(*)
from samples.high_peaks_ascents a, samples.high_peaks hp
where a.first_peak = hp.peak_name(+);
```

The Oracle syntax for an outer join requires a simple (+) for the table which is NOT the driving table in the join, so the table you need to add to your result set. When you run this query, you will notice the count is now the same as the count from the high_peaks_ascents table. You are no longer losing records by joining to the high_peaks table.

It is important for me to point out that the Oracle outer join syntax is not the ANSI SQL standard for outer joins. The same query, using the ANSI outer join syntax would look like this.

```
select count(*)
from samples.high_peaks_ascents a
LEFT OUTER JOIN samples.high_peaks hp on a.first_peak = hp.peak_name;
```

Either syntax will work, so I will accept either for your class work.

Now that we know the join is correct and we are getting the correct number of records returned from the join, let’s replace the count(*) with a few columns.

SQL Lab 2 – Table Joins

(requires access to the samples.high_peaks & samples.high_peaks_ascents tables)

```
select a.member_id, a.member_name, a.first_peak, hp.difficulty first_peak_difficulty,
a.first_ascent_date
from samples.high_peaks_ascents a, samples.high_peaks hp
where a.first_peak = hp.peak_name(+);
```

In the above query, we list a few attributes from the High_peaks_ascents table (member_id, member_name, first_peak, and ascent_date), we also list one attribute from the high_peaks table (difficulty), in order to make it clear what the difficulty rating is associated with, we have an alias for this column named first_peak_difficulty. Run the query and scroll through the result set. Member_id 16 is the first record with a First_peak value of "Unknown", you will see the "first_peak_diffulty" is null. We see all records from the high_peaks_ascents table whether or not there is a match in the high_peaks table.

Outer joins are common in ETL code, so remember to always check your record count before and after you join tables. Make sure you are getting the expected number of records returned in your result sets. For your dimensional model queries, the queries should only have simple (also known as inner) joins. It is up to the ETL process to clean up the data, to allow for clean joins against the dimensional models for the business users.

An important topic related to table joins is the Cartesian Join. If a join is not specified within an SQL statement, which contains more than one table in the FROM clause, the database will join each record from the first table to each record in the second table and repeat for all records. This results in very large and messy result sets. The number of record from the first table multiplied by the number of records from the second table.

```
select count(*)
from samples.high_peaks_ascents a, samples.high_peaks hp;
```

In the above query, I have removed the where clause, so there is no join defined, which results in a cartesian product.

Q7) What is the count for the above cartesian product?

You may include many tables in your FROM clause, but you must always include a join between each of the named tables, or the result will be a cartesian product.

Now, back to clean joins. Here is an example of joining the high_peaks_ascents table to the high_peaks table twice. The first join is on the attribute "first_peak", and the second join is on "last_peak". This would give us the ability to see the other attributes related to the peaks for either the first or last peak a member climbed. Notice that the high_peaks table is listed twice in the FROM clause. The high_peaks table also has two different aliases, the aliases are used to distinguish between the two joins and select criteria.

SQL Lab 2 – Table Joins

(requires access to the samples.high_peaks & samples.high_peaks_ascents tables)

```
select count(*)
from samples.high_peaks_ascents a, samples.high_peaks hp1, samples.high_peaks hp2
where a.first_peak = hp1.peak_name(+)
and a.last_peak = hp2.peak_name(+);
```

Q8) Does this above query return the same number of records in the result set and the number of records in the high_peaks_ascents table?

Now let's apply the concepts from this SQL tutorial. Write SQL to return the following result set.

MEMBER_ID	MEMBER_NAME	FIRST_ASCENT_DATE	FIRST_PEAK	FIRST_PEAK_RANK	FINAL_ASCENT_DATE	LAST_PEAK	LAST_PEAK_RANK
1	Herbert K. Clark	01-AUG-18	Whiteface	5	10-JUN-25	Emmons	40
2	George Marshall	01-AUG-18	Whiteface	5	10-JUN-25	Emmons	40
3	Robert Marshall	01-AUG-18	Whiteface	5	10-JUN-25	Emmons	40
4	P. F. Loope	01-MAY-27	Marcy	1	01-SEP-33	Rocky Peak	20
5	Herbert L. Malcolm	01-OCT-07	Whiteface	5	08-JUN-35	Couchsachraga	46
6	Edward C. Hudowalski	01-JUL-32	Marcy	1	13-SEP-36	Dix	6
7	Ernest R. Ryder	01-AUG-11	Whiteface	5	13-SEP-36	Dix	6
8	Orville C. Gowie	01-JUL-32	Marcy	1	25-AUG-37	Couchsachraga	46
9	Grace L. Hudowalski	02-AUG-22	Marcy	1	26-AUG-37	Esther	28
10	C. H. Nash	01-FEB-37	Marcy	1	28-AUG-37	Allen	26
11	Charles W. Horn	01-JUN-31	Marcy	1	01-SEP-37	Marshall	25
12	Henry H. Arthur	01-SEP-34	Nippletop	13	16-SEP-37	Couchsachraga	46
13	Paul H. Arthur	01-JAN-29	Marcy	1	16-SEP-37	Couchsachraga	46
14	Louise A. Goark	01-AUG-34	Colden	11	01-AUG-38	Panther	18
15	Clarence R. Craver	01-JUL-34	Street	31	01-AUG-39	Seymour	34
16	Alice Waterhouse	(null)	Unknown	(null)	08-OCT-39	Sawteeth	35
17	Ramon L. Hall	12-OCT-30	Marcy	1	14-OCT-39	Macomb	21
18	Edward A. Harmes	31-JUL-32	Whiteface	5	19-JUL-40	Couchsachraga	46
19	John M. Harmes	31-JUL-32	Whiteface	5	19-JUL-40	Couchsachraga	46
20	Edward B. Clements	01-SEP-35	Marcy	1	11-AUG-40	Allen	26
21	Rudolph Clements	01-SEP-35	Marcy	1	11-AUG-40	Allen	26
22	Nora L. Sproule	01-JAN-36	Armstrong	22	11-AUG-40	Emmons	40

Note that the screenshot only shows the first 22 records of the result set. The full result set would contain a record for each member listed in the high_peaks_ascents table.

Q9) Paste the SQL you came up with into your worksheet. Remember the column order, total number of records (in result set), and column names must match. Do not put an "order by" clause in this query. It will default to the order of the high_peaks_ascents table.

This concludes SQL Exercise 2.